

Name: Dahiwal Gajanan Manikrao
Roll No: EN24107029
Class: SY - AI \& DS
Subject: Python for Visual AI

Assignment 2

Problem Statement :Build a tool that analyzes an image's histogram and automatically adjusts brightness and contrast to produce a more balanced image.

The system should:

- Accept an image as input from the user
- Analyze pixel intensity distribution
- Apply enhancement techniques
- Display improved output image

```
In [1]: !pip install opencv-python
```

```
Requirement already satisfied: opencv-python in C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages (4.13.0.92)  
Requirement already satisfied: numpy>=2 in C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages (from opencv-python) (2.4.2)
```

```
In [ ]: import cv2
```

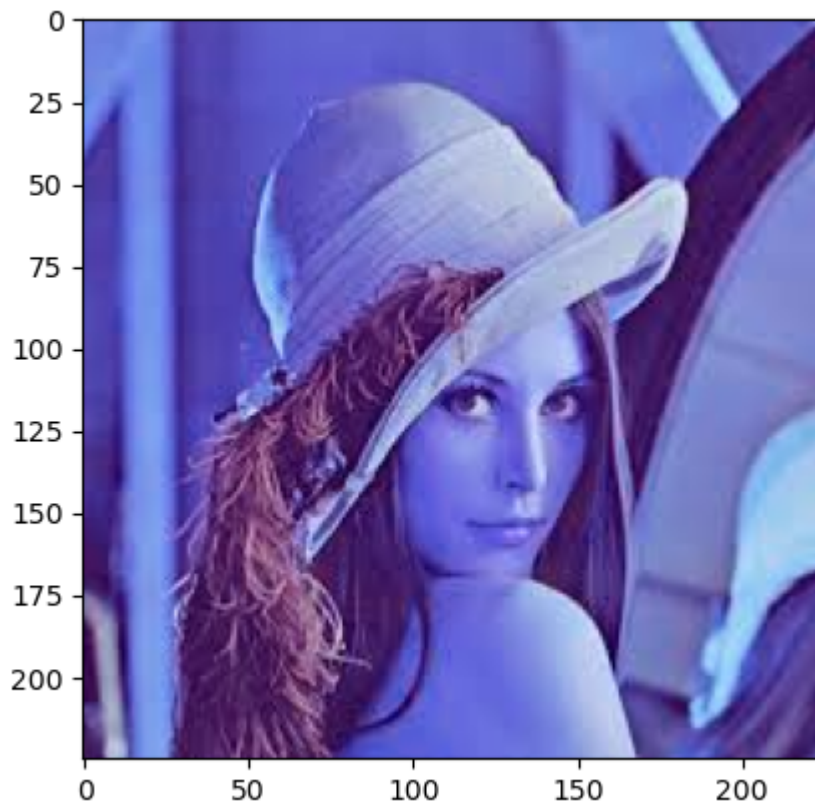
```
In [ ]: import matplotlib.pyplot as plt
```

```
In [ ]: image = cv2.imread("/home/admin1/Downloads/download.jpeg")
```

```
In [ ]: import numpy as np
```

```
In [ ]: cv2.imwrite('Original.jpg', image, [cv2.IMWRITE_JPEG_QUALITY, 100])  
plt.imshow(image)
```

```
Out[ ]: <matplotlib.image.AxesImage at 0x7faf29c3fc80>
```



```
In [ ]: r, g, b = cv2.split(image)
```

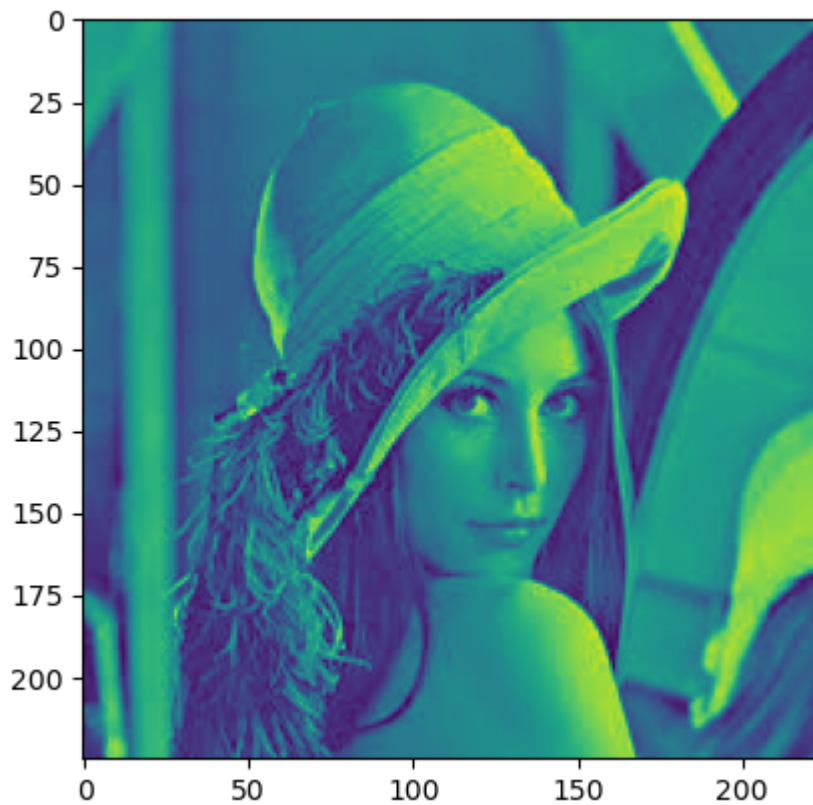
Grayscale Conversion (Luma Method)

$$luma = 0.2126R + 0.7152G + 0.0722B$$

```
In [ ]: luma = (0.2126*r) + (0.7152*g) + (0.0722*b)
cv2.imwrite('Gray_luma.jpg', luma, [cv2.IMWRITE_JPEG_QUALITY, 100])
plt.imshow(luma)
```

[WARN:0@417.598] global loadsave.cpp:1063 imwrite_ Unsupported depth image for selected encoder is fallbacked to CV_8U.

```
Out[ ]: <matplotlib.image.AxesImage at 0x7faf1d287740>
```



```
In [ ]: luma = luma.astype('uint8')
```

Image Histogram

An image histogram represents the frequency distribution of intensity levels in an image.

$$hist[i] = \text{number of pixels with intensity } i$$

```
In [ ]: hist = cv2.calcHist([luma], [0], None, [256], [0, 256])
```

```
imin = 0
for i in range(256):
    if hist[i] > 0:
        imin = i
        break

imax = 255
for i in range(255, -1, -1):
    if hist[i] > 0:
        imax = i
        break

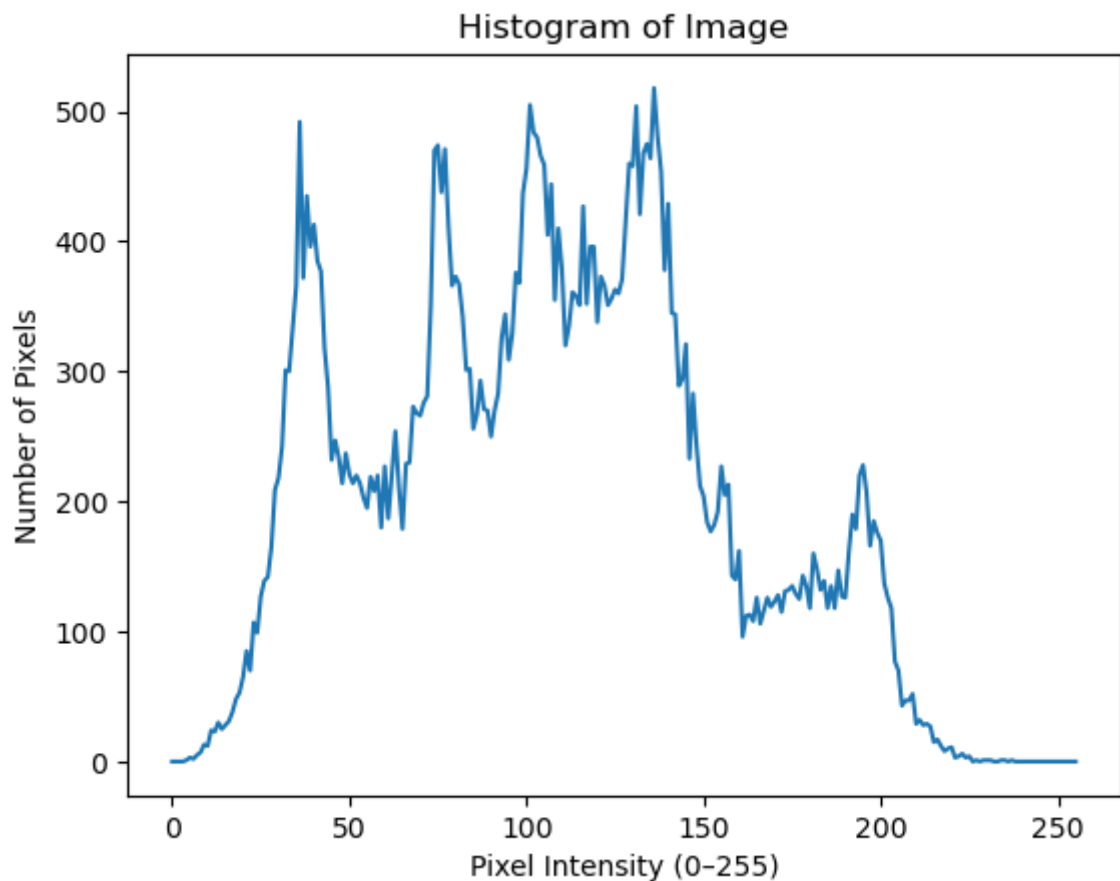
print("imin:", imin)
print("imax:", imax)
```

```
imin: 4
imax: 237
```

Histogram Analysis

- If histogram is concentrated on left → image is dark
- If histogram is concentrated on right → image is bright
- If histogram is narrow → low contrast
- If histogram is wide → high contrast

```
In [ ]: # Plot histogram
plt.plot(hist)
plt.title("Histogram of Image")
plt.xlabel("Pixel Intensity (0-255)")
plt.ylabel("Number of Pixels")
plt.show()
```



Calculation of α

$$\alpha = \frac{255}{I_{max} - I_{min}}$$

Where:

- I_{max} corresponds to `imax`
- I_{min} corresponds to `imin`

```
alpha=255/(imax-imin) print("alpha=",alpha)
```

Calculation of β

$$\beta = -\alpha \cdot I_{min}$$

Where:

- α is contrast scaling factor
- I_{min} corresponds to `imin`

```
In [ ]: beta=-alpha*imin  
print("beta=",beta)
```

beta= -4.377682403433476

Contrast Stretching Implementation

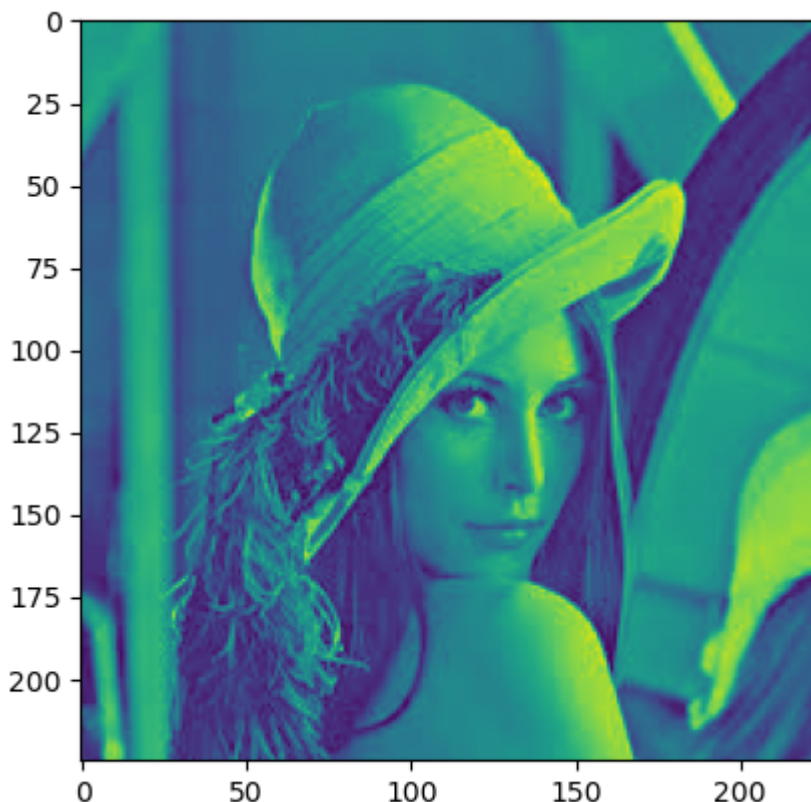
$$I_{new} = \alpha \cdot (I - I_{min})$$

Where:

- I corresponds to `luma`
- I_{min} corresponds to `imin`
- α corresponds to `alpha`
- I_{new} corresponds to `Inew`

```
In [ ]: Inew=alpha*(luma-imin)  
plt.imshow(Inew)
```

Out[]: <matplotlib.image.AxesImage at 0x7faf1c40d5e0>



Conclusion

In this assignment, histogram analysis was used to automatically adjust brightness and contrast of an image. By computing I_{min} and I_{max} , the intensity range was stretched using linear transformation. The enhanced image shows better contrast

and improved visual quality, proving that histogram-based enhancement is an effective image processing technique.